Experiment 27: Deep Belief Network Representation Learning

Question

Problem Statement: A postal service has very few labelled handwritten PIN-code digits but many unlabelled ones. It wants to pre-train features on the unlabelled data first. MNIST digits, using only 1,000 labels for fine-tuning. The objective is to build a DBN by stacking RBMs, pre-train greedily and fine-tune for classification. Use NumPy, SciPy, TensorFlow, Keras, Matplotlib.

Tasks:

1. Train two or three stacked RBMs layer by layer (784–500–250).
2. Visualise the learned weights of the first layer as image filters.
3. Transfer the weights into a Keras dense network and fine-tune with the labelled data.
4. Train the same network from random initialisation for comparison.
5. Compare accuracy and plot learned representations in 2-D.

Code

```
"""Experiment 27: Deep Belief Network Representation Learning (784-500-250, 1,000 labels)"""
import time
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from scipy.special import expit
from sklearn.manifold import TSNE

rng = np.random.RandomState(0); tf.random.set_seed(0)

def load_mnist():
    try:
        (x, y), (xt, yt) = keras.datasets.mnist.load_data()
        return x.reshape(-1, 784) / 255.0, y, xt.reshape(-1, 784) / 255.0, yt
    except Exception as e:
        print("WARNING: MNIST not available (", type(e).__name__, ") -> using scikit-learn digits scaled to
28x28.")
        from sklearn.datasets import load_digits
        from scipy.ndimage import zoom
        dg = load_digits(); d = np.stack([zoom(im / 16.0, 3.5, order=1) for im in dg.images]).reshape(-1,
784)
        cut = int(0.75 * len(d))
        return d[:cut], dg.target[:cut], d[cut:], dg.target[cut:]

x_all, y_all, x_test, y_test = load_mnist()
N_LAB = min(1000, len(x_all) // 3)
x_unl = x_all[:20000]                                          # unlabelled data for pre-training
x_lab, y_lab = x_all[:N_LAB], y_all[:N_LAB]                    # only N_LAB labels for fine-tuning
print(f"Unlabelled: {len(x_unl)} | labelled: {len(x_lab)} | test: {len(x_test)}")

class RBM:
    def __init__(self, nv, nh):
        self.W = rng.normal(0, 0.01, (nv, nh)); self.a = np.zeros(nv); self.b = np.zeros(nh)
    def p_h(self, v): return expit(v @ self.W + self.b)
    def p_v(self, h): return expit(h @ self.W.T + self.a)
    def fit(self, X, epochs=8, batch=100, lr=0.05):
        for ep in range(epochs):
            perm = rng.permutation(len(X))
            for i in range(0, len(X), batch):
                v0 = X[perm[i:i + batch]]
                ph0 = self.p_h(v0); h0 = (rng.rand(*ph0.shape) < ph0).astype(float)
                pv1 = self.p_v(h0); ph1 = self.p_h(pv1)
                self.W += lr * (v0.T @ ph0 - pv1.T @ ph1) / len(v0)
                self.a += lr * np.mean(v0 - pv1, axis=0); self.b += lr * np.mean(ph0 - ph1, axis=0)
            if ep % 2 == 1 or ep == epochs - 1:
                print(f"    epoch {ep+1}: recon MSE = {np.mean((X[:2000] - self.p_v(self.p_h(X[:2000]))) **
2):.4f}")
```

```
# ---------------- Task 1: greedy layer-wise pre-training (784 -> 500 -> 250) ----------------
t0 = time.time()
print("Pre-training RBM 1 (784 -> 500)"); rbm1 = RBM(784, 500); rbm1.fit(x_unl)
h1 = rbm1.p_h(x_unl)
print("Pre-training RBM 2 (500 -> 250)"); rbm2 = RBM(500, 250); rbm2.fit(h1)
print(f"Pre-training time: {time.time() - t0:.1f}s")


# ---------------- Task 2: first-layer filters ----------------
fig, axes = plt.subplots(8, 12, figsize=(12, 8))
for i, a in enumerate(axes.ravel()):
    a.imshow(rbm1.W[:, i].reshape(28, 28), cmap="gray"); a.axis("off")
plt.suptitle("Learned first-layer weights (RBM 1 filters)"); plt.tight_layout(); plt.show()


# ---------------- Task 3 & 4: fine-tune pretrained network vs random initialisation ----------------
def make_net():
    m = keras.Sequential([layers.Input((784,)), layers.Dense(500, activation="sigmoid"),
                          layers.Dense(250, activation="sigmoid"), layers.Dense(10, activation="softmax")])
    m.compile(optimizer=keras.optimizers.Adam(0.001), loss="sparse_categorical_crossentropy",
metrics=["accuracy"])
    return m


dbn_net = make_net()
w = dbn_net.get_weights()
dbn_net.set_weights([rbm1.W, rbm1.b, rbm2.W, rbm2.b, w[4], w[5]])    # transfer RBM weights (output layer
stays random)
rand_net = make_net()

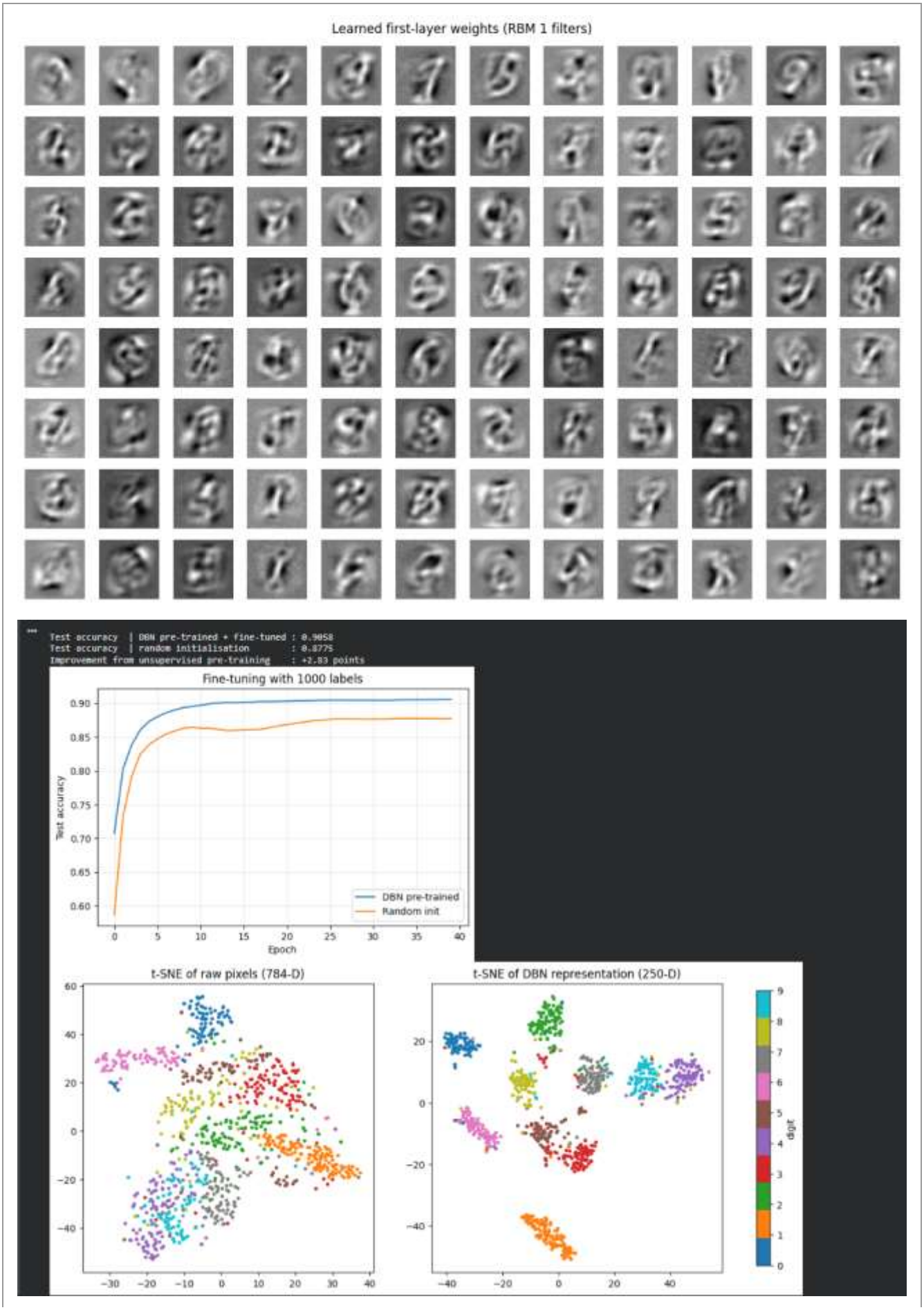

EP = 40
h_dbn = dbn_net.fit(x_lab, y_lab, validation_data=(x_test, y_test), epochs=EP, batch_size=32, verbose=0)
h_rnd = rand_net.fit(x_lab, y_lab, validation_data=(x_test, y_test), epochs=EP, batch_size=32, verbose=0)
acc_dbn = dbn_net.evaluate(x_test, y_test, verbose=0)[1]; acc_rnd = rand_net.evaluate(x_test, y_test,
verbose=0)[1]


# ---------------- Task 5: compare accuracy and plot representations ----------------
print(f"\nTest accuracy  | DBN pre-trained + fine-tuned : {acc_dbn:.4f}")
print(f"Test accuracy  | random initialisation        : {acc_rnd:.4f}")
print(f"Improvement from unsupervised pre-training    : {(acc_dbn - acc_rnd) * 100:+.2f} points")


plt.figure(figsize=(7, 4.5))
plt.plot(h_dbn.history["val_accuracy"], label="DBN pre-trained"); plt.plot(h_rnd.history["val_accuracy"],
label="Random init")
plt.xlabel("Epoch"); plt.ylabel("Test accuracy"); plt.title(f"Fine-tuning with {len(x_lab)} labels");
plt.legend(); plt.grid(alpha=0.3); plt.show()


n_vis = min(1000, len(x_test))
feat_model = keras.Model(dbn_net.inputs, dbn_net.layers[1].output)    # 250-D representation
Z_dbn = feat_model.predict(x_test[:n_vis], verbose=0)
fig, ax = plt.subplots(1, 2, figsize=(13, 5.5))
for a, data, title in zip(ax, [x_test[:n_vis], Z_dbn], ["t-SNE of raw pixels (784-D)", "t-SNE of DBN
representation (250-D)"]):
    emb = TSNE(n_components=2, random_state=0, init="pca").fit_transform(data)
    sc = a.scatter(emb[:, 0], emb[:, 1], c=y_test[:n_vis], cmap="tab10", s=8); a.set_title(title)
plt.colorbar(sc, ax=ax, fraction=0.02, label="digit"); plt.show()
```

Output


Learned first-layer weights (RBM 1 filters)
Test accuracy | DBN pre-trained + fine-tuned : 0.9058
Test accuracy | random initialisation : 0.8775
Improvement from unsupervised pre-training : +2.83 points
Fine-tuning with 1000 labels
Test accuracy
Epoch
DBN pre-trained
Random init
t-SNE of raw pixels (784-D)
t-SNE of DBN representation (250-D)
digit